

Badge Access: ≥ 3 in 1 Hour

Problem Statement

For each employee, if there exist ≥ 3 badge-ins within any rolling 60-minute window, report the window times.

Variants

- Return earliest window only vs all windows

- Include

How to Leverage the Solution

Sort times and use a sliding window; toggle output mode via early-break.

Java Solution

Below is a concise reference implementation. Full comments omitted for brevity.

```
import java.util.*;

public final class BadgeFrequencyDetector {
    public static Map<String, List<List<String>>> findAllWindows(Map<String, List<String>> logs) {
        Map<String, List<List<String>>> result = new HashMap<>();
        for (Map.Entry<String, List<String>> e : logs.entrySet()) {
            String name = e.getKey();
            List<Integer> times = new ArrayList<>();
            for (String t : e.getValue()) times.add(parse(t));
            Collections.sort(times);
            int left = 0; List<List<String>> windows = new ArrayList<>();
            for (int right = 0; right < times.size(); right++) {
                while (times.get(right) - times.get(left) > 60) left++;
                if (right - left + 1 >= 3) {
                    List<String> win = new ArrayList<>();
                    for (int k = left; k <= right; k++) win.add(format(times.get(k)));
                    windows.add(win);
                }
            }
            if (!windows.isEmpty()) result.put(name, windows);
        }
        return result;
    }

    private static int parse(String hhmm) { String[] p = hhmm.split(":"); return Integer.parseInt(p[0])
    *60 + Integer.parseInt(p[1]); }

    private static String format(int minutes) { int h = minutes/60, m = minutes%60; return String.form
    at("%02d:%02d", h, m); }
}
```